model

MovingObject Platform Terminal FixedObject Room.Color Furniture.Type Furniture.LootType RunnerRobot BlackOrb JumperRobot GameObject PuzzlePiece PlayerIsSearching ThrowerRobot Furniture AttackLaunched GameContext Room AttackerRobot.Attack LaserRobot FurnitureSearchEnded AttackEnded PlayerFoundSomething PresettedPassword RunnerStartRunning RunnerStopRunning TerminalOpened TerminalClosed PlayerDied AttackLaunced PlayerOnPlatform Leaderboard Lederboard.Entry Player javax.swing.JPanel

javax.swing.JButton

Renderer

playerSprite: PlayerSprite
context: GameContext

currentSpritesList: List<Sprite>
spritesListsCache: Map<Room, List<Sprite>>
currentWindowY: double
printSearchingState: boolean
printFurnitureLoot: boolean
interestingFurniture: Furniture
firstPaintLatch: CountDownLatch
isFirstPaint: boolean

Renderer(context: GameContext)
paintComponent(g: Graphics): void
drawHUD(g: Graphics): void
paintImage(bindedGameObject: GameObject , image: BufferedImage , g: Graphics): void
paintFurnitureInfo(furniture: Furniture , image: BufferedImage , g: Graphics): void
setCurrentSpritesList(): void
addAttackSprite(attack: AttackerRobot.Attack): void
removeAttackSprite(attack: AttackerRobot.Attack): void
removeFurnitureSprite(furniture: Furniture): void
printFurnitureLoot(furniture: Furniture): void
printSearchingState(furniture: Furniture): void
bindAllKey(context: GameContext): void
bindKey(context: GameContext , keyName: String, keyCode: int, pressed: boolean): void
setFirstPaintLatch(latch: CountDownLatch): void
getCurrentSpritesList(): List<Sprite>
getPlayerSprite(): PlayerSprite
getPreferredSize(): Dimension

PlayerSprite

PlayerSprite(player: Player)
nextImageIndex(animationList: List<BufferedImage>): int

Sprite

image: BufferedImage
gameObject: GameObject

Sprite(gameObject: GameObject)
computeImage(): void //abstract
getImage(): BufferedImage
setImage(image: BufferedImage): void
getGameObject(): GameObject

AnimatedSprite

animation: Animation

previousState: Animation.State
imageDuration: double
elapsedTime: double
imageIndex: int

AnimatedSprite(gameObject: MovingObject, animation Animation, imageDuration: double)
computeImage(): void
nextImageIndex(animationList: List<BufferedImage>): int
updateElapsedTime(deltaTime: double): void
getImageIndex(): int

Animation

PLAYER
BLACKORB
JUMPER_ROBOT

...

SEARCHING_WINDOW

animationList: Map<Animation.State, List<BufferedImage>

Animation(animationLists: Map<Animation.State, List<BufferedImage>)
getAnimationList(): ap<Animation.State, List<BufferedImage>

Animation.State

WALKING_LEFT, WALKING_RIGHT
JUMPING_LEFT, JUMPING_RIGHT

...

SEARCHING

id: int

State(id: int)
State()
getState(movingObject: MovingObject): State
isMirrored(otherState: State): boolean

BlackOrbSprite

BlackOrbSprite(blackOrb: BlackOrb)

FloorSprite

FloorSprite(floor: FixedObject, floorColor: Room.Color)
computeImage(): void

FurnitureSprite

chosenFurnitureImage: BufferedImage

FurnitureSprite(furniture: Furniture, color: Room.Color)
choseBestImage(furniture: Furniture, furnitureImages: StaticImage[]): BufferedImage
cost(furnitureWidth: double, furnitureHeight: double, imageWidth: double, imageHeight: double): double
computeImage(): void

StaticImage

GREEN_WALL_RIGHT
GREEN_WALL_LEFT
GREEN_UPPER_RIGHT_WALL

...

NUMBER_9

image: BufferedImage
color: Room.Color
type: StaticImage.Type
furnitureType: Furniture.Type
furnitureLootType: Furniture.LootType
puzzlePieceType: PuzzlePiece

StaticImage(image: BufferedImage)
StaticImge(image: BufferdImage, color: Room.Color, type: StaticImage.Type, furnitureType: Furniture.Type, furnitureLootType: Furniture.LootType , puzzlePieceType: PuzzlePiece)
StaticImge(image: BufferdImage, color: Room.Color, type: StaticImage.Type, furnitureLootType: Furniture.LootType , puzzlePieceType: PuzzlePiece)
StaticImge(image: BufferdImage, color: Room.Color, type: StaticImage.Type, furnitureType: Furniture.Type)
StaticImge(image: BufferdImage, color: Room.Color, type: StaticImage.Type, furnitureLootType: Furniture.LootType)
StaticImge(image: BufferdImage, type: StaticImage.Type, furnitureLootType: Furniture.LootType)
StaticImge(image: BufferdImage, type: StaticImage.Type)
StaticImge(image: BufferdImage, color: Room.Color, type: StaticImage.Type)
getImage(): BufferdImage
getFurniture(type: Furniture:Type, color: Room.Color): StaticImage
getFurnitures(color: Room.Color): StaticImage[]
getFloor(color: Room.Color): StaticImage
getWall(color: Room.Color, wallType StaticImage.Type): StaticImage
getPuzzlePiece(puzzlePiece: puzzlePiece): StaticImage
getFurnitureLoot(furnitureLootType: Furniture.LootType): StaticImage
getAwardsList(): List<StaticImage>
getNumbersList(): List<StaticImage>

JumperRobotSprite

JumperRobotSprite(jumperRobot: JumperRobot)
nextImageIndex(animationList: List<BufferedImage>): int

LaserRobotSprite

LaserRobotSprite(laserRobot: LaserRobot)

LaserRobotSprite.AttackSprite

AttackSprite(attack: AttackerRobot.Attack)

PlatformSprite

PlatformSprite(platform: Platform)
computeImage(): void

RunnerRobotSprite

RunnerRobotSprite(runnerRobot: RunnerRobot)

SearchingWindow

getSearchingWindow(furniture: Furniture): BufferedImage

SpriteFactory

produce(gameObject: GameObject): Sprite
produce(gameObject: GameObject, color: Room.Color): Sprite

TerminalSprite

TerminalSprite(terminal: Terminal)
computeImage(): void

ThrowerRobotSprite

ThrowerRobotSprite(throweRobot: ThrowerRobot)

ThrowerRobotSprite.AttackSprite

AttackSprite(attack: AttackerRobot.Attack)
nextImageIndex(animationList: List<BufferedImage>): int

WallSprite

WallSprite(wall: FixedObject, wallColor: Room.Color)
computeImage(): void

Menu

Menu()
paintComponent(g: Graphics): void

MenuButton

MenuButton(normalImage: BufferedImage, selectionImage: BufferedImage, width: int, height: int)

LeaderboardMenu

LeaderboardMenu(leaderboard: Leaderboard, leaderboardFont: Font)
createEntryPanel(entry: Leaderboard.Entry, leaderboardFont: Font): JPanel
paintComponent(g: Graphics): void

PlayerNameMenu

PlayerNameMenu(inputBoxFont: Font)
paintComponent(g: Graphics): void

PuzzleMenu

gridPanels: PuzzleMenu.GridPanel[]

PuzzleMenu(password: PresettedPassword, puzzlePieces: List<PuzzlePiece>, customFont: font)
createPieceLabel(puzzlePiece: PuzzlePiece): JLabel
getGridsContent(): PuzzlePiece[][][]

PuzzleMenu.GridPanel

slots: PuzzleMenu.SlotPanel[][]

GridPanel()
getSlots(): PuzzleMenu.SlotPanel[][]

PuzzleMenu.SlotPanel

SlotPanel(piecesContainer: JPanel)
isEmpty(): boolean

TerminalMenu

TerminalMenu(player: Player, terminalFont: Font)
setPositionInFrame(frameWidth: int, frameHeight: int): void
paintComponent(g: Graphics): void

MenuEvent

«record»
CloseGame

«record»

GamePanelRequested

playerName: String

«record»

InputBoxMenuRequested

«record»

LeaderboardMenuRequested

«record»

PlatformResetRequested

«record»
PuzzleMenuClosed

«record»

PuzzleMenuOpened

«record»

PuzzleMenuRequested

player: Player

«record»

RobotDisableRequested

«record»

SecondaryMenuClosed

«record»
TerminalClosed

ImageUtils

loadImage(pathname: String, crop: boolean): BufferedImage
loadFlipped(pathname: String, crop: boolean): BufferedImage
loadAnimation(pattern: String, start: int, end: int, crop: boolean): List<BufferedImage>
loadFlippedAnimation(pattern: String, start: int, end: int, crop: boolean): List<BufferedImage>
loadRaw(pathname: String): BufferedImage
flipHorizontally(image: BufferedImage): BufferedImage
crop(image: BufferedImage): BufferedImage
imageStrip(image: BufferedImage, times: int, direction: ImageUtils.Direction): BufferedImage
scaleImage(image: BufferedImage, width: int, height: int): BufferedImage
getNumberAsImagesList(number: int, symbolsList: StaticImage[]): List<BufferedImage>

ImageUtils.Direction

VERTICAL
HORIZONTAL

StaticImage.Type

TERMINAL, MIDDLE_WALL_RIGHT, MIDDLE_WALL_LEFT, UPPER_LEFT_WALL,
UPPER_RIGHT_WALL, BOTTOM_LEFT_WALL, BOTTOM_RIGHT_WALL, FURNITURE, FLOOR, PLATFORM,
FURNITURE_LOOT

AudioBank

BACKGROUND_MUSIC
EMPTYLOOT_SAMPLE

...

PLAYERDEATH_SAMPLE

audioData: AudioData

AudioBank(audioData: Audiodata)
getAudioData(): AudioData

«record»
AudioData

audioContent: byte[]
audioFormat: AudioFormat

AudioPlayer

instance: AudioPlayer
backgroundClip: Clip
runningClips: Map<Object, Clip>
playBackgroundMusic: void

getInstance()
AudioPlayer()
playSample(audioSample: AudioData): void
startRunningClip(object: Object, audioSample: AudioData): void
removeRunningClip(object: Object): void
stopAllRunningClips(): void
startAllRunningClips(): void
disposeBackgroundMusic(): void
disposeRunningClips(): void

AudioUtils

loadAudioSample(pathname: String): AudioData
toClip(audioSample: AudioData): Clip
loadRaw(pathname: String): AudioData